Generalized Sudoku with SAT: Comparing Three At-Most-One Encodings and CP/ILP Baselines

Tạ Đình Nguyên

INT3301E_4, Student ID: 24021587,

ABSTRACT

This study represents generalized Sudoku with cell–value Boolean variables and compares pairwise, sequential-counter, and bitwise At-Most-One encodings using the same Glucose3 solver. The dataset contains 60 synthetic instances at grid sizes 4, 9, 16, and 25 and three clue densities, plus 20 public 9×9 instances. CP-SAT/HiGHS are evaluated on a 16-instance subset; Gurobi, CPLEX MIP, and CP Optimizer on a 10-instance subset. Three repetitions and a 10-second cutoff covering model construction/loading and solving produce 906 runs: 903 independently checked solutions and three HiGHS timeouts on one instance. All three SAT encodings solve all 80 instances in every repetition. On five synthetic 25×25 instances with 30% clues, median total times for pairwise, sequential, and bitwise are 0.402449, 0.159527, and 0.210650 seconds; build costs exceed solving costs. However, rankings change with grid size; CPLEX MIP has a lower median than pairwise on four common public instances. These results emphasize construction costs and comparisons on identical instance sets, without establishing a universally superior solver or encoding. Uniqueness remains unverified and baseline coverage is limited.

Student: Tạ Đình Nguyên, ID 24021587, course INT3301E_4. Instructor: TS. Tô Vân Khánh. Confirmed submission deadline: 23:59 on 19 October 2026, Vietnam time.

1. Introduction and related work

Sudoku has concise rules, yet the same feasibility problem can be represented in SAT, CP, or ILP. As grid size increases, representations can differ in variable counts, constraints, and model construction costs. Evaluation therefore needs to distinguish representation size, construction/loading time, and solving time; fewer clauses alone do not establish faster solving. This work considers generalized Sudoku with $n \geq 2$ and $N = n^2$, focusing on three AMO encodings within a common SAT formulation.

1.1. Related foundations

Lynce–Ouaknine formulate Sudoku using cell–value Boolean variables and distinguish minimal and extended models [11]. Their work supplies modeling background, not results for this project's dataset. Sinz presents a sequential unary counter for cardinality constraints [14]. PySAT supplies a toolkit and the CardEnc API for AMO choices [10, 12]. Here, ALO and the four EXO families stay fixed while AMO varies among pairwise, seqcounter, and bitwise; bitwise AMO is not identified with Sinz's binary counter. These sources support construction methods, not correctness checks or measurements of the current program.

1.2. Research questions

1. How do the three AMO encodings affect primary/auxiliary variables, clauses, and build/solve/total times as grid size and clue density vary?
2. On matching instance sets, how does SAT compare with CP-SAT and HiGHS ILP in valid-solution coverage and time under the same cutoff?
3. What additional evidence do available Gurobi/CPLEX comparisons on the small subset provide, and how do runtime/license limits restrict coverage?

S80/B16/C10 specify comparison subsets before measurement. Section 9 answers these questions using checked measurements and the correct intersections; SAT statistics for all of S80 are not combined with B16/C10 baseline statistics as though they used the same sample.

1.3. Scope of the contribution

The course-project contribution is an implementation of generalized Sudoku models, integration of established encodings under a common specification, model/solution validation, and a reproducible evaluation through 906 runs on defined comparison sets. No new encoding is proposed, existing runtimes/libraries are not presented as algorithms invented by the author, and no superiority is claimed for all n or all Sudoku instances.

2. Problem definition and notation

Let $n \geq 2$ be an integer and $N = n^2$. The $N \times N$ Sudoku grid contains N blocks of size $n \times n$. Rows and columns are zero-based; cell values belong to $D = \{1, \ldots, N\}$. Input uses 0 for an empty cell. A solution completes the grid, preserves its clues, and contains each value exactly once in every row, column, and block. Uniqueness is not required.

Define $I = \{0, \dots, N-1\}$, $J = \{0, \dots, n-1\}$, and the nonzero clue triples $G \subseteq I \times I \times D$, with $g = |G|$. The Boolean variable $x_{r,c,v}$ is true exactly when cell (r, c) contains value v. There are N^3 primary variables. The implementation maps them to

$$\mathrm{id}(r, c, v) = (rN + c)N + v \ \in \{1, \dots, N^3\}. \tag{1}$$

The parser rejects malformed structure or domains as INVALID_INPUT. Structurally valid conflicting clues are still encoded; infeasibility becomes UNSAT only when proved. Figure 1 illustrates $n = 2$.

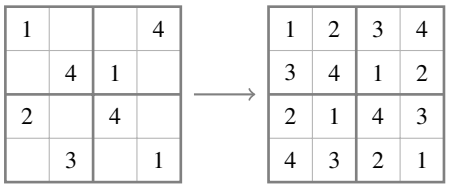


Figure 1: A 4×4 input (left) and a constructed completion (right), for illustration; not solver output or a uniqueness certificate.

3. SAT formulation and three AMO encodings

The cell–value representation and extended formulation of Lynce–Ouaknine provide background [11]. For a group $Z = (z_1, \dots, z_m)$, Exactly-One decomposes into ALO and AMO:

$$\mathrm{EXO}(Z) = \mathrm{ALO}(Z) \wedge \mathrm{AMO}(Z), \qquad \mathrm{ALO}(Z) = \bigvee_{i=1}^{m} z_i. \tag{2}$$

The following four families each contain N^2 groups of length N. Together with the clue units they form the Sudoku formula:

$$\bigwedge_{r,c \in I} \mathrm{EXO}\{x_{r,c,v} : v \in D\}, \qquad \text{cell;} \tag{3}$$

$$\bigwedge_{r \in I, v \in D} \mathrm{EXO}\{x_{r,c,v} : c \in I\}, \qquad \text{row–value;} \tag{4}$$

$$\bigwedge_{c \in I, v \in D} \mathrm{EXO}\{x_{r,c,v} : r \in I\}, \qquad \text{column–value;} \tag{5}$$

$$\bigwedge_{p,q \in J, v \in D} \mathrm{EXO}\{x_{pn+u,qn+w,v} : u, w \in J\}, \qquad \text{block–value;} \tag{6}$$

$$\bigwedge_{(r,c,v) \in G} x_{r,c,v}, \qquad \text{clues.} \tag{7}$$

Thus there are $4N^2$ EXO groups. Code uses the same group/literal order for every encoder, appends ALO before AMO within each group, and then appends clues in row-major order. Duplicate clauses remain; no symmetry breaking or clue-specific preprocessing is applied.

3.1. Pairwise

$$E_{\mathrm{pw}}(Z) = \bigwedge_{1 \le i < j \le m} (\neg z_i \vee \neg z_j). \tag{8}$$

Every input pair is forbidden from being simultaneously true. Code generates these pairs directly without auxiliaries.

3.2. Sequential counter

Following the sequential unary counter principle [14], AMO can be specified using a chain $s_1, \dots, s_{m-1}$:

$$\begin{aligned} E_{\mathrm{sc}}(Z, S) = {} & (\neg z_1 \vee s_1) \wedge (\neg z_m \vee \neg s_{m-1}) \\ & \wedge \bigwedge_{i=2}^{m-1} \left[(\neg z_i \vee s_i) \wedge (\neg s_{i-1} \vee s_i) \wedge (\neg z_i \vee \neg s_{i-1}) \right]. \end{aligned} \tag{9}$$

A true input propagates along the chain, excluding any later true input. For an extension, choose $s_i = \bigvee_{j \le i} z_j$. The program calls `CardEnc.atmost` with `bound=1`, `EncType.seqcounter`, and the current `top_id` [12, 10]. The expression specifies the principle and semantics; it does not claim byte-for-byte literal order or auxiliary naming in the PySAT binary.

3.3. Bitwise AMO

Let $k = \lceil \log_2 m \rceil$ and assign each input a distinct binary code $b(i) \in \{0,1\}^k$. With selectors $a_1, \ldots, a_k$, let $\ell_{ij} = a_j$ if $b(i)_j = 1$, and $\ell_{ij} = \neg a_j$ otherwise. Then

$$E_{\mathrm{bw}}(Z, A) = \bigwedge_{i=1}^{m} \bigwedge_{j=1}^{k} (\neg z_i \vee \ell_{ij}). \tag{10}$$

Two true inputs require distinct codes and therefore conflict on at least one bit. With one true input, choose its code; with no true inputs, selectors are arbitrary. The program uses `EncType.bitwise` through the same AMO API. This encodes AMO over one-hot literals; it does not replace Sudoku cell variables by binary numbers. The bitwise API/source is credited separately [12, 13], not attributed to Sinz.

4. Correctness

Let $F_e(X, A)$ be the CNF for encoding e, including clues. Each AMO must satisfy $\exists A_Z\, E_e(Z, A_Z) \iff \sum_i z_i \le 1$, as the constructions above do. Code reserves all N^3 primary IDs first, allocates each group's auxiliaries above `top_id`, and updates it, so groups do not share auxiliaries. Thus

$$\exists A\, F_e(X, A) \quad \iff \quad \mathrm{Sudoku}_G(X). \tag{11}$$

Forward direction. Cell ALO and AMO give exactly one value per cell. Row–value, column–value, and block–value groups require every value exactly once in the corresponding unit. Unit clauses preserve every clue. Decoding a CNF model's primary variables therefore yields a valid solution.

Reverse direction. From a Sudoku solution preserving clues, set the selected cell–value variables true. Every group has exactly one true input. Pairwise is already satisfied; extend sequential groups by prefix ORs and bitwise groups by the true input's code. Disjoint auxiliary sets let these local extensions combine into a global satisfying assignment A. This does not require every auxiliary assignment to satisfy the CNF, nor imply uniqueness.

5. Theoretical size and checked counts

For $m \ge 4$, the presented AMO constructions have the following auxiliary counts $a_e(m)$ and clause counts $c_e(m)$:

e	$a_e(m)$	$c_e(m)$
pairwise	0	$m(m-1)/2$
seqcounter	$m-1$	$3m-4$
bitwise	$\lceil \log_2 m \rceil$	$m\lceil \log_2 m \rceil$

(12)

Including one ALO per group, the theoretical totals are

$$V_e = N^3 + 4N^2 a_e(N), \qquad C_e = 4N^2[1 + c_e(N)] + g. \tag{13}$$

Pairwise has exactly N^3 variables and $4N^2[1 + N(N-1)/2] + g$ clauses with duplicates retained. Clause growth for the constructions is respectively $\Theta(N^4)$, $\Theta(N^3)$, and $\Theta(N^3 \log N)$; all have $\Theta(N^3)$ total variables, since $g \le N^2$. These describe representation size, not solve-time bounds or evidence of a faster encoder.

Table 1 is generated from `diagnostics/model/results.json`: PySAT 1.9.dev16, $N \in \{4, 9, 16, 25\}$. These are pre-preprocessing CNF counts, not benchmarks. Diagnostic cases have $g = 2$; the table subtracts the two clue units to show $C - g$. Observed seqcounter/bitwise counts match the expressions at these four sizes. This does not assert identical counts for every variant, library, or input length; small groups may use API shortcuts. The upstream source read is not treated as proof of the running binary.

Table 1
Structural counts from diagnostics; $C - g$ excludes clue units.

N	AMO	N^3	Aux.	V_e	$C_e - g$
4	pairwise	64	0	64	448
4	seqcounter	64	192	256	576
4	bitwise	64	128	192	576
9	pairwise	729	0	729	11988
9	seqcounter	729	2592	3321	7776
9	bitwise	729	1296	2025	11988
16	pairwise	4096	0	4096	123904
16	seqcounter	4096	15360	19456	46080
16	bitwise	4096	4096	8192	66560
25	pairwise	15625	0	15625	752500
25	seqcounter	15625	60000	75625	180000
25	bitwise	15625	12500	28125	315000

6. CP and ILP comparison models

The comparison models describe the same task: finding one completion that preserves the clues, not optimizing puzzle difficulty or solution quality. This section describes the accepted builders; successful construction does not establish successful solving or license coverage at every size.

6.1. CP with AllDifferent

Using the previously defined I, J, D, G, introduce an integer variable $z_{r,c} \in D$ for each cell and impose

$$\text{AllDifferent}(z_{r,c} : c \in I), \qquad r \in I, \tag{14}$$

$$\text{AllDifferent}(z_{r,c} : r \in I), \qquad c \in I, \tag{15}$$

$$\text{AllDifferent}(z_{pn+u,qn+w} : u, w \in J), \qquad p, q \in J, \tag{16}$$

$$z_{r,c} = v, \qquad (r, c, v) \in G. \tag{17}$$

Each unit has N cells with an N-element domain, so AllDifferent forces each value to occur exactly once. There are N^2 integer variables and $3N + g$ model-level constraints; these are not clause/variable counts after internal solver transformations.

The CP-SAT builder creates a `CpModel`, a fresh `CpSolver`, and sets one worker [4]. The CP Optimizer builder uses the same domains, AllDifferent constraints, and clues with `Workers=1` [9]; its executable is checked separately. The accepted builder package does not itself solve, decode, or validate a solution.

6.2. Common binary ILP

Let $y_{r,c,v} \in \{0, 1\}$ for $r, c \in I$ and $v \in D$. Use four equality families and the clues:

$$\sum_{v \in D} y_{r,c,v} = 1, \qquad r, c \in I, \tag{18}$$

$$\sum_{c \in I} y_{r,c,v} = 1, \qquad r \in I,\ v \in D, \tag{19}$$

$$\sum_{r \in I} y_{r,c,v} = 1, \qquad c \in I,\ v \in D, \tag{20}$$

$$\sum_{u,w \in J} y_{pn+u,qn+w,v} = 1, \qquad p, q \in J,\ v \in D, \tag{21}$$

$$y_{r,c,v} = 1, \qquad (r, c, v) \in G. \tag{22}$$

The builder exports a CSR matrix M, an all-ones right-hand side, and an all-zero objective:

$$\min\ \mathbf{0}^{\mathsf{T}} Y = 0 \quad \text{subject to} \quad MY = \mathbf{1}, \quad 0 \le Y \le 1, \quad Y \in \mathbb{Z}^{N^3}. \tag{23}$$

All feasible solutions have objective 0, so this is a feasibility problem. The objective does not minimize execution time. Zero-based columns use $\mathrm{id}(r, c, v) - 1$. The model has N^3 binary variables, $4N^2 + g$ equality rows, and $4N^3 + g$ nonzeros before preprocessing. Each EXO group/clue produces one row; potentially redundant rows remain, with no backend-specific domain reduction or pruning.

The same ILP structure is used by Gurobi, CPLEX MIP, and HiGHS adapters. Execution, thread/budget settings, statuses, and checker integration are described separately in Section 7; builder evidence alone does not establish runtime coverage. At the model level, $y_{r,c,v} = 1 \iff z_{r,c} = v$ yields the same Sudoku solutions as the SAT representation, despite different internal solver representations.

7. Algorithm and adapter implementation

The pseudocode below summarizes the implemented parser, model builder, solver adapter, decoder, and separate checker. Earlier equations specify the models; the actual source code belongs in the repository. This workflow does not replace a runner with a hard process guard or establish that a benchmark has run.

Pseudocode: solve one input

1. Read/parse the input into a Puzzle; structural, JSON, or file errors produce INVALID_INPUT. Check method/encoding/budget against the API before building.
2. Prepare backend libraries outside measurement; report missing runtime as UNAVAILABLE.
3. Start the monotonic clock at t_0; use the earlier of t_0+cutoff and any caller deadline. If the budget is exhausted, return TIMEOUT.
4. Build the CNF or CP/ILP model and create/load a fresh solver. Account for build/setup; if the budget is exhausted, skip solving and return TIMEOUT.
5. Call the solver with the remaining budget, using supported native limits and cooperative interruption. Retain the raw status.
6. Record native-solve completion and observed component times. Late results become TIMEOUT; do not admit a late incumbent.
7. Return UNSAT only on proved infeasibility. With no acceptable candidate, classify TIMEOUT, ERROR, or LICENSE_LIMIT from evidence; do not turn unknown into UNSAT.
8. Decode the candidate into a complete grid; decoding failure is ERROR. SAT decodes primary variables only; MIP values must pass finite/integrality checks before selection.
9. Run the independent checker on domains, rows, columns, blocks, and clues. Publish SAT with a solution only if accepted; rejection or checker failure is ERROR.
10. On every exit path, cancel/join timers before native solver deletion and release resources. Report cleanup failure explicitly rather than publishing a successful solution.

Figure 2: Adapter workflow, not evidence of a runner hard cutoff.

7.1. Library use and validation

The SAT backend is `pysat.solvers.Glucose3` in python-sat 1.9.dev16 [10, 1, 2]. The five baselines use OR-Tools CP-SAT [4], gurobipy [5], the direct CPLEX Python API (`cplex.Cplex`, without DOcplex.MP) [8], DOcplex.CP for CP Optimizer [9], and highspy for HiGHS [6, 7]. In these adapters SciPy supplies sparse ILP matrices; it is not a HiGHS solve backend through `scipy.optimize.milp`. For SAT, the public API consists of `prepare_backend` and `solve_sat`; each call uses a fresh Glucose3 instance. The adapter calls the CNF builder with the selected encoding, uses interrupt-capable native solving, then passes the model through decoding and `validation.check_solution`. Auxiliary variables do not determine decoded cell values.

For comparisons, `solve_baseline` accepts one of `cpsat`, `gurobi`, `cplex_mip`, `cpoptimizer`, or `highs`. CP-SAT/CP Optimizer use the CP builders; Gurobi/CPLEX MIP/HiGHS consume the same ILP matrix, bounds, integrality, and constant objective. Each call has fresh objects and one thread/worker. Adapters retain raw statuses and use the same production checker.

MIP decoding accepts only finite values within 10^{-6} of 0 or 1, with exactly one selected value per cell; arbitrary rounding of fractional solutions is not allowed. HiGHS must also report acceptable primal feasibility/integrality before decoding. A time-limit incumbent can become SAT only if it arrived on time and passes checking. UNSAT requires infeasibility evidence; missing runtime, license limits, and unknown outcomes remain distinct.

7.2. Timing boundary and process safeguards

Imports/input reading precede measurement. Build includes model/CNF construction and solver creation/loading; solving uses the remaining budget. Decoding, checking, and cleanup are outside build+solve. Deadline admission uses native-solve completion, not checker completion as the solve endpoint. Timing components unknown after forced termination must remain missing rather than invented.

SAT native interruption and baseline timers/native limits are cooperative. In DOcplex 2.32.264, TimeLimit is embedded in transmitted CPO, so an exact post-load native TimeLimit is not promised: the adapter uses preload for build accounting, an abort_search timer against the actual deadline, and late-result rejection. The private preload hook needs rechecking on DOcplex upgrades. If timer joining fails, the adapter reports ERROR and may defer native deletion to avoid a callback into released memory.

The measurement-end hook and hard worker/process-tree guard are accepted. The runner uses START/END to apply the cutoff to measurement, then switches to a postprocessing watchdog and process-tree cleanup as needed. The pilot exercised this integration; native timeout alone remains cooperative. Section 8 describes the frozen main configuration; Section 9 reports its executed results separately from the pilot.

8. Experimental setup

The main batch `main-20261016-01` completed 906 fresh runs with a 10-second cutoff. Dataset, source code, configuration, and dependencies are frozen; `run.json` and `execution-freeze.json` retain metadata/hashes reconciled with the final analysis audit. Results in Section 9 belong only to this main batch.

8.1. Machine and runtimes

Runs are configured sequentially on the same Windows x64 machine, one job at a time and one thread/worker per engine. Table 2 distinguishes bindings from native runtimes where evidence exists. The native Glucose3 version is not inferred from PySAT, nor SciPy's embedded HiGHS from highspy.

Table 2
Machine and versions in the frozen main-run metadata.

Component	Recorded value
OS	Windows 11, 10.0.26300, x64
CPU	AMD Ryzen 7 H 260 w/ Radeon 780M Graphics; 16 logical CPUs
RAM	33,618,251,776 bytes (≈ 31.31 GiB)
Python	3.13.5
SAT	python-sat 1.9.dev16; Glucose3 (no separate native-version API in the probe)
CP-SAT	OR-Tools 9.15.6755
HiGHS	highspy 1.15.1; native 1.15.1
Gurobi	gurobipy 13.0.3; native 13.0.3
CPLEX MIP	cplex 22.2.0.1; native 22.2.0.0
CP Optimizer	docplex 2.32.264; native 22.2.0.0
Sparse matrices	SciPy 1.18.1

8.2. Data and comparison subsets

S80 comprises 60 synthetic instances: four sizes $N = 4, 9, 16, 25$, three target clue densities of 30%, 50%, and 70%, and five instances per group; plus 20 public 9×9 instances. Actual clue counts and seeds are in the frozen manifest. Public instances come from one Sudoku Exchange hard.txt file at commit `d8c8ebaee0c08c412cfba96af1923dfa61c83317` [15], sampled with seed 20261016 in the saved sample order.

B16 is chosen before results: sort each synthetic size–density group by actual seed and then instance ID and take the first (12 instances), plus the first four public instances in the fixed sample order. C10 is the $4 \times 4/9 \times 9$ part of B16: three synthetic instances at each size and the same four public instances. Thus $C10 \subset B16 \subset S80$. SAT runs three encodings on S80; CP-SAT/HiGHS use B16; Gurobi/CPLEX MIP/CP Optimizer use C10. SAT is not rerun separately for subsets. The main configuration has three repetitions per instance–configuration pair: $720 + 96 + 90 = 906$ runs, with a schedule shuffled using seed 20261016 and saved before execution. The pilot executed 55 jobs at repetition one to check integration/coverage, not 906 runs or three main repetitions. Pilot SAT/timeout rates and timings are not main Results.

8.3. Timing and process safeguards

Worker imports/input reading precede START. START/END use synchronized shared state and a monotonic clock. From START, the 10-second cutoff includes model/CNF construction, solver creation/loading, and solving; solving uses only the remaining budget. On-time END switches to a separate postprocessing watchdog. Decoding, validation, cleanup, and CSV/output I/O are outside build+solve. Missing or late END causes TIMEOUT; unknown timing components remain missing.

The Windows runner assigns the worker to a kill-on-close Job Object before releasing execution, terminates the process tree on deadline, and waits for descendants. The startup watchdog is 60 seconds before START; postprocessing has 30 seconds after on-time END. Startup/postprocessing/cleanup failure is ERROR, distinct from measurement timeout. Watchdogs do not extend solve time. The hard guard and measurement-end hook passed the pilot before use in the 906 fresh main runs.

CP Optimizer uses DOcplex 2.32.264 private preload to charge compile/send/load to build. Native TimeLimit is embedded in CPO before loading; after loading, cooperative abort_search is backed by the external guard. Exact post-load native TimeLimit changes are not promised; DOcplex upgrades require rechecking the private hook as in Section 7.

8.4. Aggregation rules

For each instance–configuration pair with three SAT runs, take the three-run median separately for build, solve, and total, then take the median of per-instance medians within the specified stratum. Components are aggregated independently: per-run total equals build+solve, with setup included in build, but the two group medians need not sum to the group total median. Paired runtime comparisons use only IDs with three SAT runs on both sides and report matched counts; solved counts and timeouts cover the full selected scope. Per-run PAR-2 uses SAT total or $2 \times 10 = 20$ seconds for TIMEOUT; ERROR, INVALID_INPUT, UNAVAILABLE, and LICENSE_LIMIT are excluded. The fastest repetition is not selected, and missing timings are not filled with zero.

9. Results and discussion

The analysis uses only the main batch `main-20261016-01`: 906 runs, 903 SAT outcomes accepted by the independent checker, and 3 TIMEOUT outcomes; there are no UNSAT, ERROR, INVALID_INPUT, LICENSE_LIMIT, or UNAVAILABLE outcomes. Pilot data are excluded. Table 3 reports both runs and instances with three SAT outcomes within each scope.

Table 3
Coverage of the 906 main runs.

Configuration	Scope	SAT/runs	Instances with 3 SAT	TIMEOUT
Glucose3 pairwise	S80	240/240	80/80	0
Glucose3 sequential	S80	240/240	80/80	0
Glucose3 bitwise	S80	240/240	80/80	0
CP-SAT	B16	48/48	16/16	0
HiGHS ILP	B16	45/48	15/16	3
Gurobi MIP	C10	30/30	10/10	0
CPLEX MIP	C10	30/30	10/10	0
CP Optimizer	C10	30/30	10/10	0

9.1. CNF size and construction cost

Figure 3 fixes clue density at 50% to isolate grid-size effects; vertical axes are logarithmic, variable counts include auxiliaries, and counts precede preprocessing. Seed 0 is representative because counts agree across all five seeds within each size–density stratum. Table 4 separately illustrates N25/30% with 188 clues: pairwise has no auxiliaries but the most clauses; sequential has more auxiliaries than bitwise but fewer clauses. CNF counts are not proxies for native CP/MIP model sizes.

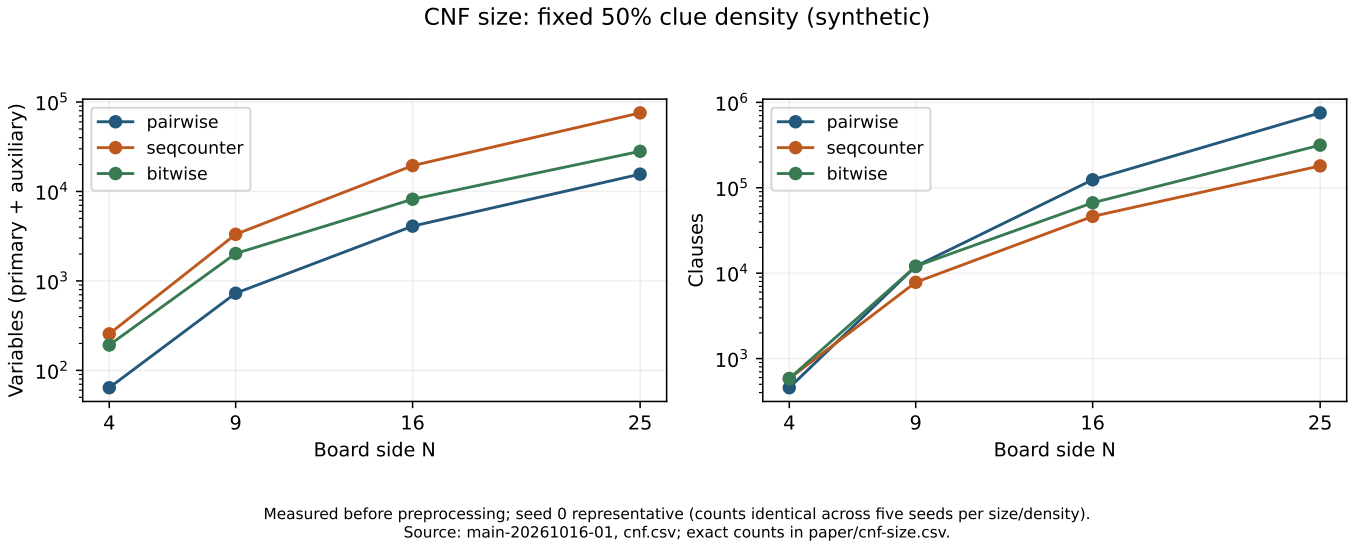


Figure 3: Synthetic CNF size at fixed 50% clue density; seed 0 at each size, before preprocessing.

Table 4
Measured CNF for synthetic-N25-p30-s0, 188 clues; before preprocessing.

Encoding	Primary	Auxiliary	Total variables	Clauses
pairwise	15625	0	15625	752688
sequential	15625	60000	75625	180188
bitwise	15625	12500	28125	315188

Table 5 uses the same five IDs, `synthetic-N25-p30-s0` through `s4`, each with three SAT runs for all encodings. Build, including solver creation/loading, exceeds solve time in this stratum. Sequential has the lowest median total even though bitwise has a lower median solve time. Thus solve-only timing or variable counts omit a substantial pipeline cost.

Table 5
Synthetic 25×25, 30% clues: five common IDs, seconds.

Encoding	Median build	Median solve	Median total
pairwise	0.381882	0.022380	0.402449
sequential	0.130810	0.024383	0.159527
bitwise	0.193860	0.015416	0.210650

Each column is an independently aggregated median of per-instance three-run medians; the group build and solve medians do not sum to the group total median.

Figure 4 separates the three densities; each point is the median of five per-instance medians, each based on three repetitions. Rankings are not fixed: for synthetic 4×4/30%, pairwise/sequential/bitwise median totals are 0.001189/0.001520/0.001582 seconds. Sequential's lowest total at N25/30% does not imply the lowest time for every n or dataset.

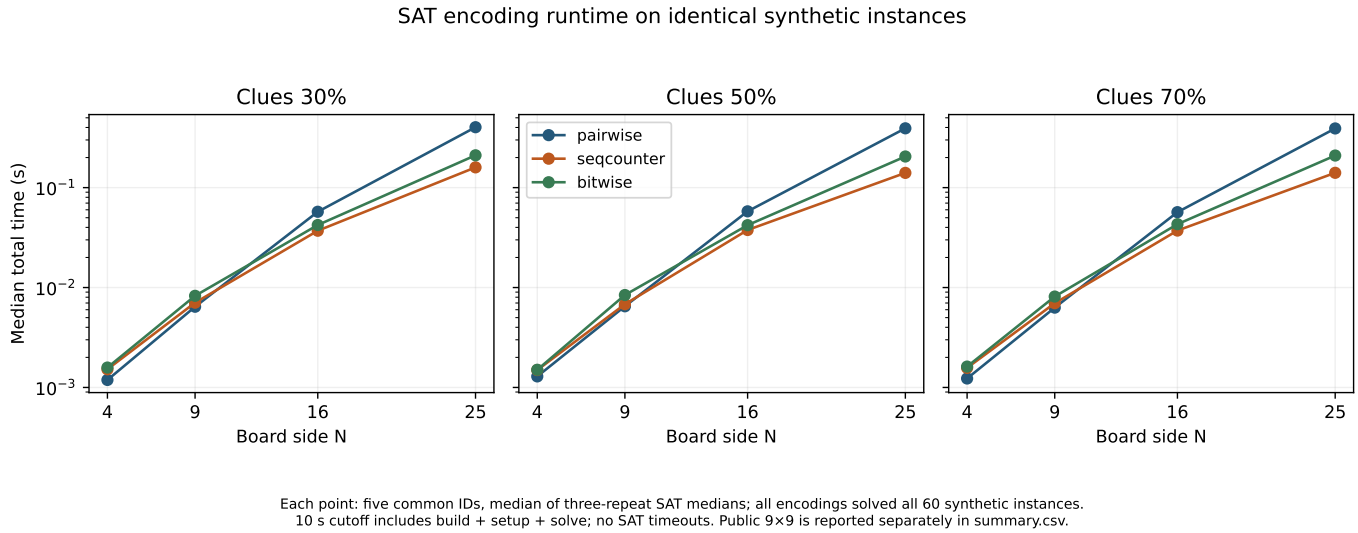


Figure 4: SAT median total (seconds), five common synthetic IDs per point; three repetitions/ID, with a 10-second cutoff for build (including setup) + solve. No SAT encoding times out.

9.2. Baselines on matching instance sets

Tables 6–8 come from `paper/baseline-matched.csv`, using pairwise as a fixed reference rather than choosing the fastest encoding separately for each opponent. Median totals include only IDs with three SAT runs in both configurations. In B16 each density at N25 has one ID, unlike the five-ID S80 statistic in Table 5. For example, on `synthetic-N25-p30-s0`, CP-SAT takes 0.230552 seconds versus pairwise's 0.394866 seconds; the latter must not be replaced by the five-ID median of 0.402449 seconds.

Table 6
B16 synthetic: median total on each common ID, seconds.

N	Clues (%)	m_{CP}/m_H	CP-SAT	HiGHS	Pairwise
4	30	1/1	0.004866	0.001704	0.001184
4	50	1/1	0.004223	0.001638	0.001453
4	70	1/1	0.004099	0.001468	0.001323
9	30	1/1	0.013557	0.012148	0.006350
9	50	1/1	0.006484	0.004400	0.006544
9	70	1/1	0.005491	0.004467	0.006700
16	30	1/1	0.057879	0.047280	0.057086
16	50	1/1	0.009205	0.021862	0.058682
16	70	1/1	0.007168	0.022792	0.055755
25	30	1/0	0.230552	–	0.394866
25	50	1/1	0.055806	0.113672	0.382179
25	70	1/1	0.012724	0.066873	0.383525

Here m counts IDs with three SAT runs in both configurations. Each row has one ID, unlike five IDs per S80 stratum. A dash denotes three HiGHS timeouts at N25/30%, with no SAT median.

Table 7
C10 synthetic: one common ID per row, median total (seconds).

N	Clues (%)	m	Gurobi	CPLEX MIP	CP Optimizer	Pairwise
4	30	1	0.002521	0.002137	0.029427	0.001184
4	50	1	0.002477	0.002280	0.033032	0.001453
4	70	1	0.002210	0.002362	0.030086	0.001323
9	30	1	0.010647	0.006728	0.029006	0.006350
9	50	1	0.007375	0.003787	0.029938	0.006544
9	70	1	0.007339	0.003884	0.030118	0.006700

Table 8
Four common public 9×9 instances in B16/C10, median total (seconds).

Backend	Scope	m	Backend	Pairwise
CP-SAT	B16	4	0.006506	0.006534
HiGHS ILP	B16	4	0.004704	0.006534
Gurobi MIP	C10	4	0.008019	0.006534
CPLEX MIP	C10	4	0.004748	0.006534
CP Optimizer	C10	4	0.030911	0.006534

The same four IDs are L069940, L293964, L269904, and L215069 (prefix public-se-hard-). Pairwise is a fixed reference, not the fastest encoding selected separately for each opponent.

On the same four public instances, CPLEX MIP has a median total of 0.004748 seconds, below pairwise's 0.006534 seconds; Gurobi at 0.008019 and CP Optimizer at 0.030911 seconds are higher than pairwise. This counters an assertion that SAT always wins; it does not establish that ILP or one solver is always best. Ratios are not reported in these tables; a paired ratio would require the median of per-ID ratios rather than division of group medians. Millisecond differences on a small sample require caution.

9.3. Timeouts, PAR-2, and traceability

All three timeouts are the three HiGHS repetitions of `synthetic-N25-p30-s0`. This ID has no SAT median; the runtime intersection between HiGHS and SAT/CP-SAT on B16 contains 15 IDs, while its intersection with commercial engines on C10 still contains 10. Two SAT encodings share 80 IDs. Each timeout receives PAR-2 = $2 \times 10 = 20$ seconds, for a total timeout penalty of 60 seconds; mean PAR-2 for the HiGHS N25/30% stratum is 20 seconds. Missing timings are not zero: r3 lacks build/solve/total and remains missing, while r1/r2 have observed totals near the cutoff but remain TIMEOUT. Successful-only runtime statistics must be read with the counts in Table 3. Tables and figures use the locked final analysis; `summary.csv`, `instances.csv`, `matched-pairs.csv`, and `cnf.csv` retain detailed statistics. `paper/raw-row-index.csv` maps job IDs to original CSV rows; the audit retains configuration, versions, and hashes. Pilot data are not merged, and the benchmark is not rerun to select favorable results.

10. Limitations

Synthetic instances originate from structured valid grids and transformations, introducing generator bias rather than representing all Sudoku. Larger grids or fewer clues do not establish greater difficulty. Public instances are also externally machine-generated, with only 20 sampled from one hard.txt bucket; bucket names/ratings are uncalibrated source labels. The 80 instances have SAT evidence but unverified uniqueness; finding one valid completion is not a uniqueness certificate.

Baselines are supplementary small samples: compare three SAT encodings on S80, CP-SAT/HiGHS on B16, and commercial engines on C10 or the available intersection. Paired runtimes use instances with three successful repetitions in both configurations and disclose scoped counts/coverage. Medians from different sets are not a common-sample comparison. A 10-second timeout is cutoff-censored, not true solving time. ERROR, UNAVAILABLE, and LICENSE_LIMIT are distinct from TIMEOUT and excluded from timeout PAR-2 penalties.

Free editions restrict coverage: Gurobi size-limited allows 2,000 variables/2,000 linear constraints; CPLEX MIP Community allows 1,000/1,000. The $4 \times 4/9 \times 9$ C10 models fit and completed all three main repetitions on C10; this does not imply all-S80 coverage. CP Optimizer has a model/search-space-dependent limit: a size failure on one 16×16 or 25×25 grid does not establish failure for every grid of that size. HiGHS is not relabeled as Gurobi/CPLEX.

One machine, one version configuration, three repetitions, and a finite cutoff do not establish generality across hardware/solvers. B16 has one ID per synthetic stratum; C10 contains only small grids and four common public instances. Millisecond differences are sensitive to overhead; no statistical significance testing, RAM measurement, or search counters are provided. Baselines use models and engines different from SAT, so differences cannot all be attributed to encoding. Smoke/pilot are separate integration gates and are excluded from main results.

11. Conclusion

The cell–value model with four EXO families permits AMO substitution under a common Sudoku specification. Pairwise, sequential-counter, and bitwise encodings with the same Glucose3 solver solve all 80 instances in all three repetitions. On five synthetic N25/30% instances, sequential has a median total of 0.159527 seconds versus pairwise's 0.402449 and bitwise's 0.210650 seconds; build accounts for most cost in this stratum. This answers the encoding question through variable/clause tradeoffs and construction/loading cost, not solving alone. Rankings change on small grids, so no universal winning encoding is selected. CP-SAT/HiGHS on B16 and commercial solvers on C10 provide scoped comparisons: HiGHS times out in all three runs of one ID, while CPLEX MIP has a lower median than pairwise on four common public instances. These small samples do not establish that SAT, CP, or ILP always wins. Future work should broaden instance sources, verify uniqueness and difficulty, increase repetitions and machines, and examine runtime costs further while retaining matched subsets and cutoff-aware treatment.

Code and data availability

Source code, configuration, data, result CSVs, freeze metadata, and analysis accompany the local handover package. The main batch is `results/raw/main-20261016-01`; tables/figures and audit are in `results/summary/main-20261016-01`. Public data retain Sudoku Exchange provenance/license [15]; reused-component notices accompany the source package. The report uses cas-sc and Tectonic [3, 16]. Source code, data, and results are published at `https://github.com/Ngynkybyrd/2627I_INT3011E`. The immutable initial snapshot is commit `f45d37894cfb19fe3bb890e8f0e06f13b472c2a2`, containing source/data/results verified against the local copy. This report was finalized after that snapshot; course submission remains deferred.

References

[1] Audemard, G., Simon, L., 2009. Predicting learnt clauses quality in modern SAT solvers, in: Proceedings of the Twenty-First International Joint Conference on Artificial Intelligence, pp. 399–404. URL: https://www.ijcai.org/Proceedings/09/Papers/074.pdf.

[2] Eén, N., Sörensson, N., 2004. An extensible SAT-solver, in: Theory and Applications of Satisfiability Testing, Springer. pp. 502–518. URL: https://doi.org/10.1007/978-3-540-24605-3_37, doi:10.1007/978-3-540-24605-3_37. proceedings of SAT 2003; publisher citation year 2004.

[3] Elsevier, 2024. Elsevier CAS Templates. Elsevier. URL: https://ctan.org/pkg/els-cas-templates. version 2.4; template provenance, not a research reference.

[4] Google OR-Tools Team, n.d. CP-SAT Solver. Google. URL: https://developers.google.com/optimization/cp/cp_solver. living documentation; accessed 16 October 2026. Recorded OR-Tools distribution: 9.15.6755.

[5] Gurobi Optimization, LLC, 2026. Gurobi Optimizer Reference Manual. URL: https://docs.gurobi.com/projects/optimizer/en/current/reference/python/model.html. year follows the vendor's explicit 2026 citation recommendation; accessed 16 October 2026. Recorded gurobipy and native runtime: 13.0.3.

[6] HiGHS Developers, n.d. HiGHS Documentation: Python Examples. URL: https://ergo-code.github.io/HiGHS/dev/interfaces/python/example-py/. living documentation; accessed 16 October 2026. The recorded local highspy runtime is 1.15.1.

[7] Huangfu, Q., Hall, J.A.J., 2018. Parallelizing the dual revised simplex method. Mathematical Programming Computation 10, 119–142. URL: https://doi.org/10.1007/s12532-017-0130-5, doi:10.1007/s12532-017-0130-5. preferred citation recommended by the HiGHS project; not a description of its full MIP algorithm.

[8] IBM, n.d.a. CPLEX Python API Reference Manual: Meet the Python API. URL: https://ibmdecisionoptimization.github.io/cplex-python-doc/manual/03_meet_python.html. documentation banner 22.1.2; accessed 16 October 2026. API reference only: the installed cplex distribution is 22.2.0.1 and recorded native runtime is 22.2.0.0; not DOcplex.MP.

[9] IBM, n.d.b. Creating a Constraint Programming Model (DOcplex.CP). URL: https://ibmdecisionoptimization.github.io/docplex-doc/cp/creating_model.html. documentation banner V2.32; accessed 16 October 2026. Recorded DOcplex interface: 2.32.264; CP Optimizer executable runtime: 22.2.0.0.

[10] Ignatiev, A., Morgado, A., Marques-Silva, J., 2018. PySAT: A Python toolkit for prototyping with SAT oracles, in: Theory and Applications of Satisfiability Testing – SAT 2018, Springer. pp. 428–437. URL: https://doi.org/10.1007/978-3-319-94144-8_26, doi:10.1007/978-3-319-94144-8_26.

[11] Lynce, I., Ouaknine, J., 2006. Sudoku as a SAT problem. Author-hosted paper, listed in Proceedings of AIMATH 06. URL: https://people.mpi-sws.org/~joel/publications/sudoku05.pdf. 9-page author version; author landing page lists AIMATH 06 (2006), with copyright 2005.

[12] PySAT Developers, n.d.a. Cardinality Encodings (pysat.card). PySAT. URL: https://pysathq.github.io/docs/html/api/card.html. documentation banner 1.9.dev16; accessed 16 October 2026.

[13] PySAT Developers, n.d.b. PySAT cardinality encoder source: bitwise.hh. URL: https://github.com/pysathq/pysat/blob/218c1c163f837c6512d8970e4bdf49ae02cee779/cardenc/bitwise.hh. upstream commit 218c1c163f837c6512d8970e4bdf49ae02cee779; header credits the card library from BOLT; accessed 16 October 2026. Not yet matched to the installed wheel.

[14] Sinz, C., 2005. Towards an optimal CNF encoding of boolean cardinality constraints, in: Principles and Practice of Constraint Programming – CP 2005, Springer. pp. 827–831. URL: https://www.carstensinz.de/papers/CP-2005.pdf, doi:10.1007/11564751_73.

[15] Sudoku Exchange, n.d. Sudoku Exchange Puzzle Bank: hard.txt. GitHub repository grantm/sudoku-exchange-puzzle-bank. URL: https://github.com/grantm/sudoku-exchange-puzzle-bank/tree/d8c8ebaee0c08c412cfba96af1923dfa61c83317. commit d8c8ebaee0c08c412cfba96af1923dfa61c83317; dataset public-domain dedication; accessed 16 October 2026. Uniqueness is a source claim, not independently verified.

[16] Tectonic Project, n.d. The Tectonic Typesetting System: Installation. Tectonic Project. URL: https://tectonic-typesetting.github.io/book/latest/installation/. accessed 16 October 2026; compiler release 0.17.0.